Computer Graphics: Lecture 13

Meshes, Topologies, and Indices

Slide Deck © Grant Williams, 2026, License: CC-BY-SA 4.0

Welcome back!

Last time, we learned about interactive cameras.

We learned how to move SceneNodes around.

We learned how to invert the camera's model matrix to make a view matrix.

We learned about Tait-Bryan/Euler angles, and the yaw-pitch-roll camera system. We also learned about alternatives.

We learned how to read keyboard input.

We learned what a keycode is.

This time

This lecture is all about meshes.

We've been using them the whole time, but we haven't really used them the way they get used in actual 3D engines.

So let's change that! We're going to make a mesh class, learn how to describe faces, and learn how to load meshes from a file.

What is a mesh?

A mesh is a list of triangles representing an object's boundary surface.

The boundary surface is basically a surface that separates the inside of the object from the outside.

We've been learning all about meshes, because every thing we want to draw in 3D has a mesh.

Meshes are not the only way to draw objects. For example, voxel systems exist, where we sample points inside the region of space occupied by the object and draw those. But modern graphics systems usually convert even those into meshes anyway.

What's in a mesh?

A mesh contains a list of points.

But not only that, or else it would be a point cloud. It also is a list of triangles.

You've seen many meshes in your exposure to 3D graphics.

Here's one in which the triangles are particularly visible.



Final Fantasy VII

Meshes are important

Because a mesh represents something we want to draw, it is a super important concept.

However, the way we've been drawing meshes is rather cumbersome.

Specifically, we've been storing meshes as a list of vertices.

The triangles have been implicit. Every three vertices has been grouped together into a triangle.

This works, but it means we have to repeat a lot of vertices. Typically, a triangle will share an edge with another triangle, meaning two repeated vertices each time that happens.

Indices

To streamline this, almost all the time a real 3D engine uses a mesh, or a 3D modelling program exports a mesh, it uses **indices**.

An **index** is an integer that refers to a vertex.

The mesh is then described by two pieces of information:

1. A list of vertices
2. A list of indices

Let's see a simple example...

An indexed triangle

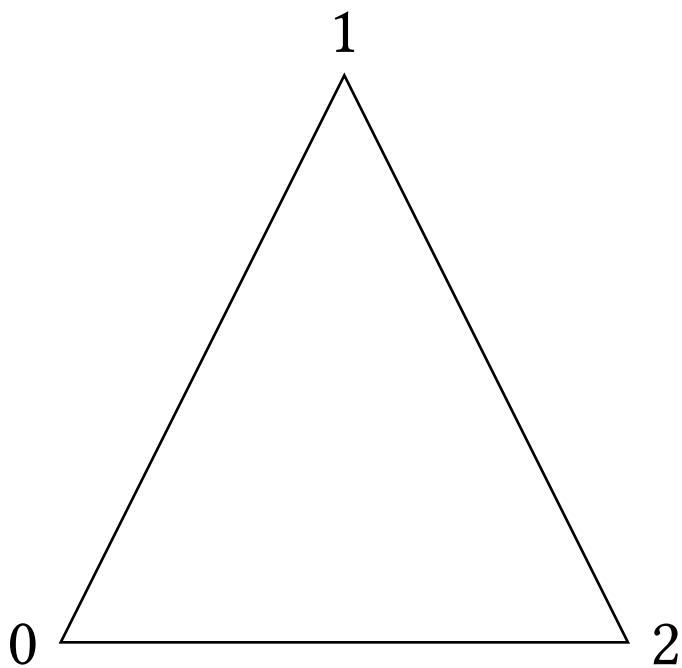
Here are some vertices that describe a triangle:

```
const vertData = new Float32Array([
  -.75, -.75, 0,
     0,  .75, 0,
   .75, -.75, 0,
]);
```

We can use indices too:

```
const indis = new Uint32Array([2, 1, 0])
```

This says “draw a triangle from points 2, 1, and 0” (which is important: 0, 1, 2 would be clockwise)

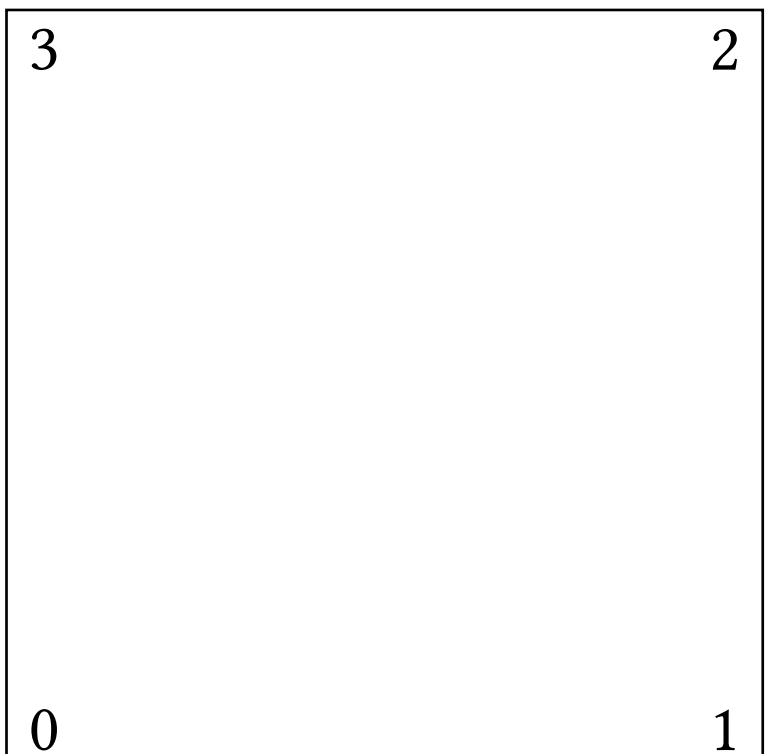


But why?

Cool, we can take the thing we were already doing and now add 3 numbers to it. What's the point?

The point is that this ends up saving time and space when we have triangles that share edges.

For example, a quad: `[0, 1, 2, 0, 2, 3]`



The quad

Notice: before when we wanted a quad, we needed to specify 6 vertices.

That means 6 positions, not to mention all the other attributes (6 colors, 6 uv-coordinates, in the future 6 vertex normals).

Now we only need 4, and then an index buffer of integers.

The indices can get grouped together in groups of 3 like before, so we draw the triangle 0, 1, 2, and then the triangle 0, 2, 3.

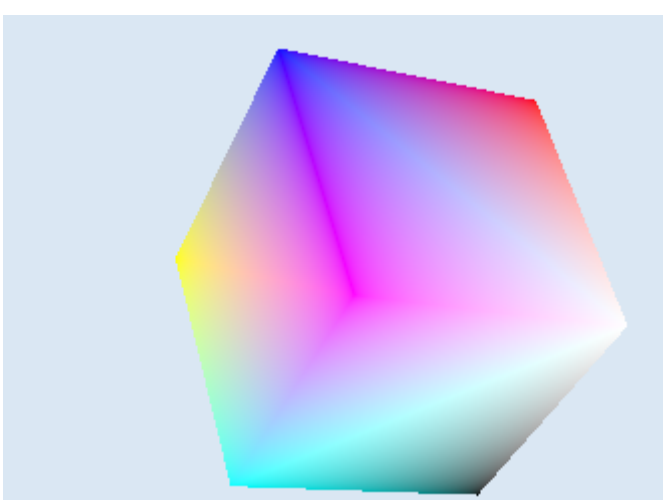
There are other ways to group indices, which we will discuss soon.

What if we made a cube?

The cube

We can define a cube with 8 vertices.

How many indices does it take? At maximum, it would take 36. Each face of the cube has 2 triangles, so that's 6 indices per face, and there are 6 faces. Much nicer than needing 36 vertices.



The cube (2)

Just be aware, this cube only needs 8 vertices, but others might need more.

For example, if you had a separate piece of texture on each side of the cube, you would need to duplicate vertices to give new UV coordinates to each face.

You can only share vertices if all the data (including all the attributes, and not just position) is exactly the same.

Luckily, most of the time, it actually is.

More complicated meshes

A more complex mesh can really save a lot of space using index lists.

The GPU is designed to work with them efficiently.

In fact, we can get much more efficient than you've seen, using different **topologies**, but that will come soon.

So, hopefully you're sold on index buffer's usefulness.

Questions?

Drawing a mesh *without* indices

Previously, here is how we drew a simple mesh:

1. Create a vertex buffer and load it with the mesh's data
2. Create a shader module that is compatible with the vertex data.
3. Create a pipeline, and reference the shader module and attributes.
4. Create a command encoder and a pass.
5. Set the pipeline and vertex buffer.
6. Draw

Drawing a mesh *with* indices

To use indices, there are a few things we need to adjust:

- In addition to the vertex buffer, we also create an index buffer. It is a buffer with `GPUBufferUsage.INDEX` instead of `.VERTEX`. We use integer data, so instead of `Float32Array` we use `Uint32Array` as the source data.
- In the pass, we don't only set the vertex buffer. We also set the index buffer: `pass.setIndexBuffer(indexBuffer, 'uint32');` The `uint32` is there because some index buffers use smaller `uint16`s to save space.
- When we draw, we use `pass.drawIndexed(indexCount);` instead of regular `pass.draw(vertexCount)`.

That's all

You might think there would need to be more. Adjusting the shaders, more pipeline configuration, bind group stuff, but no, that's it.

We add an additional buffer for the indices, we set it, and then use a different method to draw.

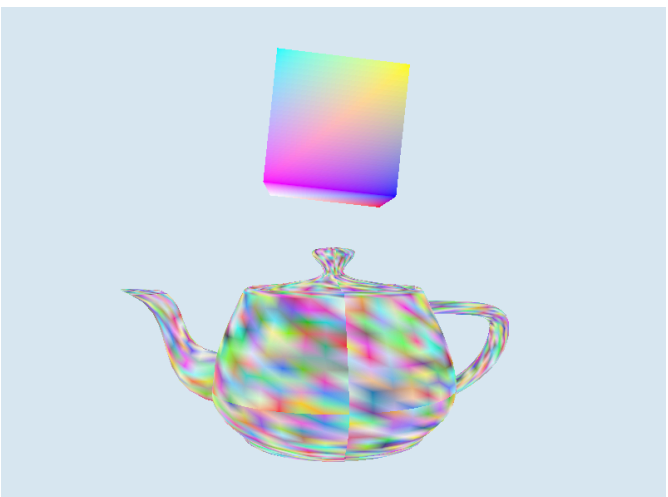
Just to be clear: you don't need to mess with the shaders at all. They don't know whether they are being called on indices or vertices. The vertex shader still gets the vertex data. The primitive assembly system still constructs triangles for the fragment shader to interpolate over.

However, the vertex shader can be called fewer times now. It won't need to be called on duplicate vertices.

The sample

Let's look at the sample to see an example.

Notice, there is a spinning cube and…is that a teapot?



(don't mind the colors for the teapot, I'm generating them randomly)

The mesh classes

Two items of interest: there are two mesh classes.

One of them is for describing a mesh (its colors, positions, and indices):

```
export class SimpleMesh {
    indices: number[];
    positions: vec3[];
    colors: vec3[]; // we could add more attributes easily
    nVerts: number;
    topology: GPUPrimitiveTopology; // (covered soon)
    stride: number = 6 * 4; // 3 position floats, 3 colors
    name: string;
    ...
}
```

The mesh classes (2)

The second class stores the info needed to *draw* the mesh. That is, the data on the GPU itself, after the mesh is loaded.

```
class LoadedSimpleMesh {
    vertexBuffer: GPUBuffer;
    indexBuffer: GPUBuffer;
    nVerts: number;
    nIndis: number;
    topology: GPUPrimitiveTopology;
    ...
}
```

The mesh classes (3)

These classes are useful because they group together the two buffers we're going to need: vertex and index.

They also keep track of how much data is in them: something we used to hardcode.

But what's that topology thing?

Topologies

The topology describes how the indices group vertices together.

The default topology is `"triangle-list"`, which means that we're describing a list of triangles, three vertices at a time.

Technically there are also options for lines (useful for wire-frame debuggging) and points (for point clouds).

But there's another option, called `"triangle-strip"`...

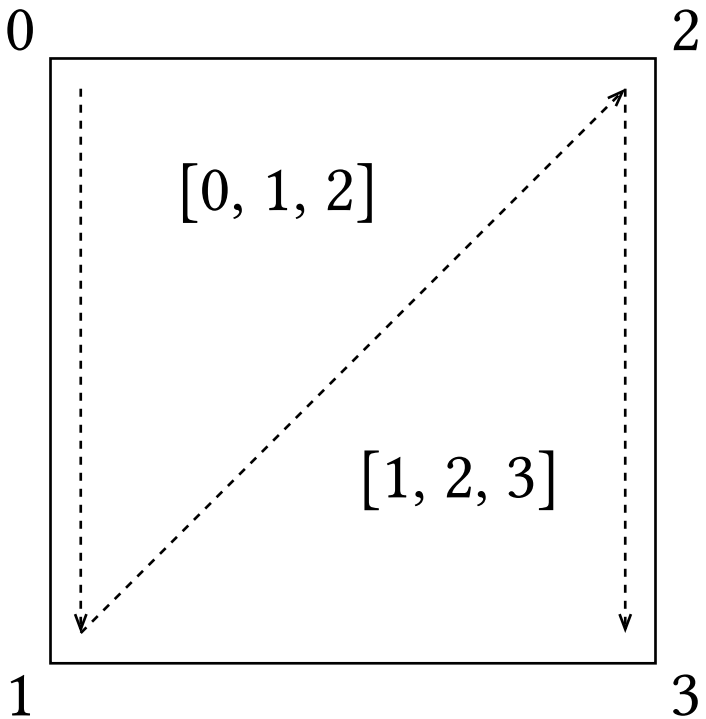
Triangle strips

The vast majority of the time, the triangles are not independent. Instead, they usually are tesselating a larger figure, meaning a bunch of triangles are sharing edges.

It's wasteful to describe a triangle that shares an edge with another triangle. We end up specifying the vertices multiple times.

For example, instead of the triangle list `[0, 1, 2, 2, 1, 3]`, we could describe it as a triangle strip: `[0, 1, 2, 3]`...

Triangle strips (2)



A quad drawn from the triangle strip [0, 1, 2, 3]

Triangle strips (3)

When the GPU encounters a triangle, it forms a triangle from the first three indices. In the previous slide that was [0, 1, 2].

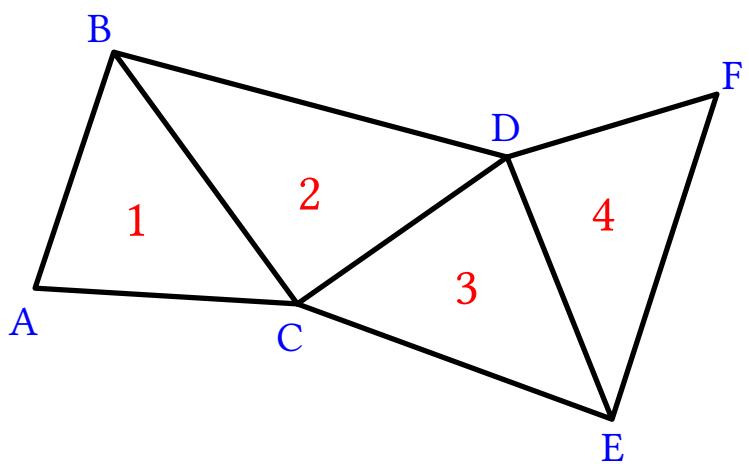
Then, every index that follows shares the edge from the previous two indices. So the 3 ends up being linked with the 1, 2: [1, 2, 3]

What about winding order, though? [0, 1, 2] is counter-clockwise, but [1, 2, 3] is clockwise.

Don't worry, the GPU handles that. Every other triangle in a strip has its winding order inverted. So it just knows that the [1, 2, 3] should be facing the same way as [0, 1, 2].

Triangle strips (4)

There's no reason we have to stop at a quad. We can draw a whole ribben of indices in a triangle strip, approaching a two-thirds index savings…



a clockwise strip: [A, B, C, D, E, F]

Drawing more than a ribbon…

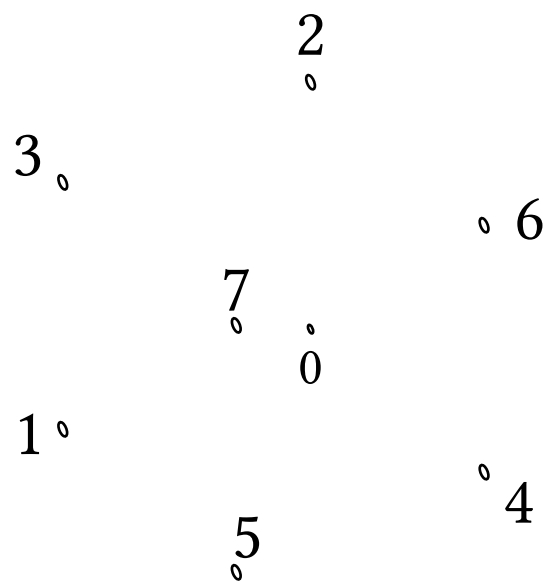
In fact, drawing ribbons is the main point. It's a "strip" after all.

But what happens when we reach the end of a ribbon?

For example, suppose we want to draw an extremely common shape: a cube. Let's try drawing it as a triangle strip…

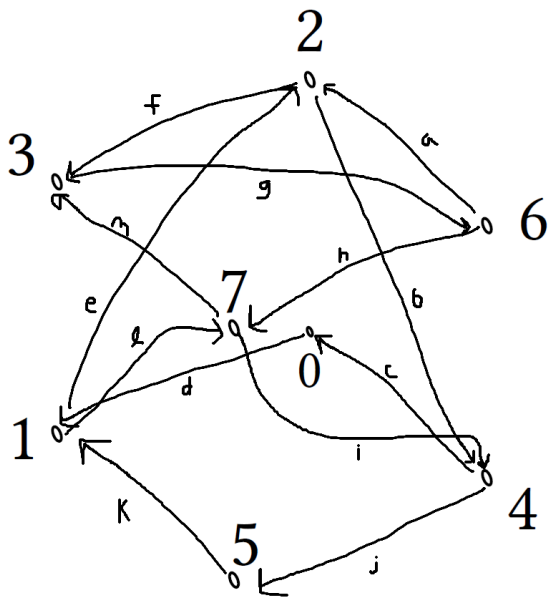
Attempting a triangle-strip cube

Here are 8 vertices:



Make me a cube with a triangle strip! The indices don't have to go in order! But be sure the winding is correct.

It's actually possible...



The strip is: [6, 2, 4, 0, 1, 2, 3, 6, 7, 4, 5, 1, 7, 3]

Credit to this paper and this stackoverflow

…but it's hard

When you need to look up a research article to tesselate the cube, you might get a bit discouraged.

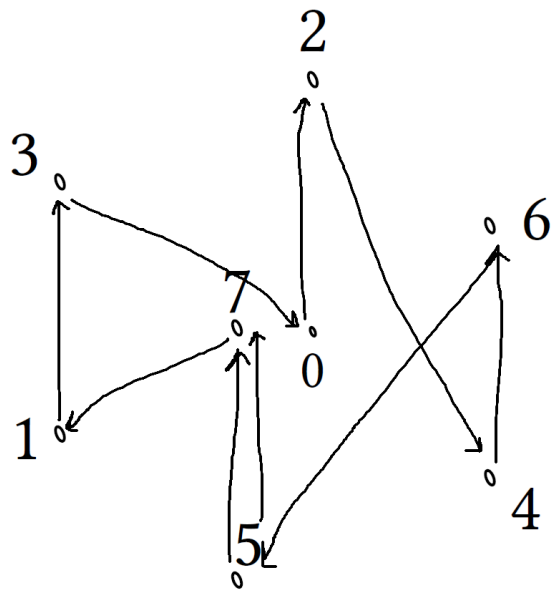
And for some meshes it's *actually* impossible. For example, meshes that are disconnected. There's literally no way to do it.

Luckily, there's a useful feature: **primitive restart**.

It's a special index, equal to the largest integer supported by your index size. If you're using `Uint16`s, then it's `0xFFFF`. If you're using `Uint32`s, it's `0xFFFFFFFF`.

When the GPU sees that, it *snips* (restarts) the triangle strip.

Restarted cube



The strip is: [5, 7, 1, 3, 0, 2, 4, 6, 5, 7, `0xFFFF`, 3, 7, 2, 6, `0xFFFF`, 5, 1, 4, 0]

I left out the top and bottom faces in the drawing, but you can see them after the primitive restart indices (`0xFFFF`s).

Not as good?

This strip isn't quite as optimized. It ends up requiring more indices.

But it's a bit easier to understand.

To see a similar strip in action, look at `simpleCubeMesh()` in `sample10`.

Using triangle strips

So, is there anything we need to change to use triangle-strips? How does WebGPU know our topology?

Look at the `primitive` field of the pipeline descriptor:

```
primitive: {
    cullMode: 'back',
    frontFace: 'ccw',
    topology: 'triangle-strip',
    stripIndexFormat: 'uint16',
}
```

We changed the topolgy and also told it what kind of integer will be used. The only options are `"uint16"` and `"uint32"`.

Anything else?

That's it. Your existing index-buffer will work, you'll just store a strip in it instead.

If you use `uint32`, the restart index is `0xFFFFFFFF` instead.

That's all for now regarding topologies. For the next part, we're going to go back to normal triangle lists. But we *will* be using index buffers pretty much from now on.

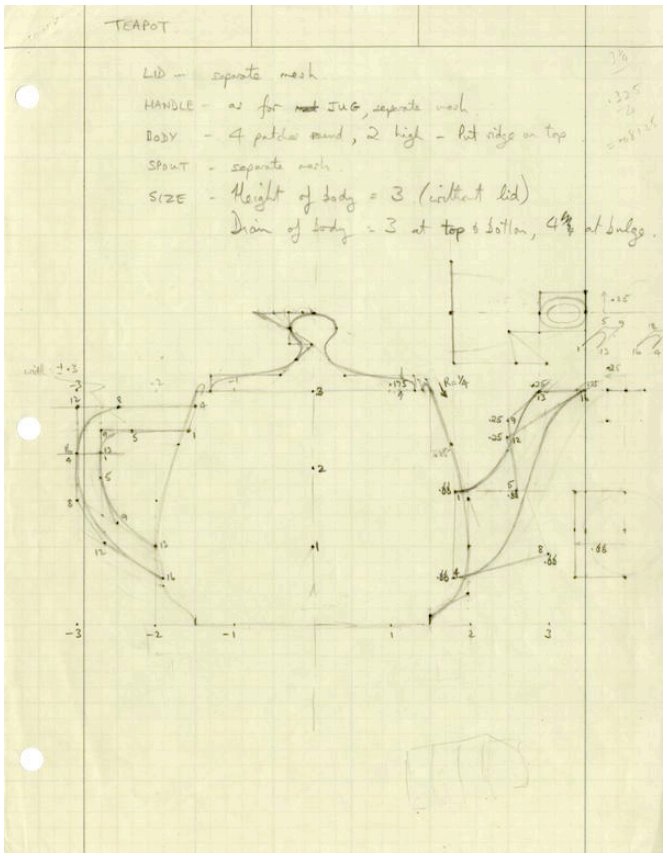
Questions

The teapot

Okay, what's the deal with the teapot?

It's the Utah Teapot, named after the University of Utah where the early computer graphics researcher Martin Newell painstakingly derived the bezier curves of a Melitta brand teapot.

It's a computer graphics in-joke now, and is often a test model.



The teapot (2)

The teapot is a *real* mesh. Not a cube or a triangle, but an actual boundary surface of a 3D object.

But how do we draw it? There's no `pass.drawTeapot()` method in WebGPU. Though maybe there should be.[1]

We're going to have to load this one from a file! But what kind of file?

[1] You might think I'm joking but some graphics libraries literally have a draw teapot function. It's that popular. It's useful to test lighting.

Obj files

We're going to use an `.obj` file!

No, not the object files your C compiler spits out. That has the same name but it's completely different.

This file format refers 3D objects. It's also a text format, so it will be easier for us to parse.

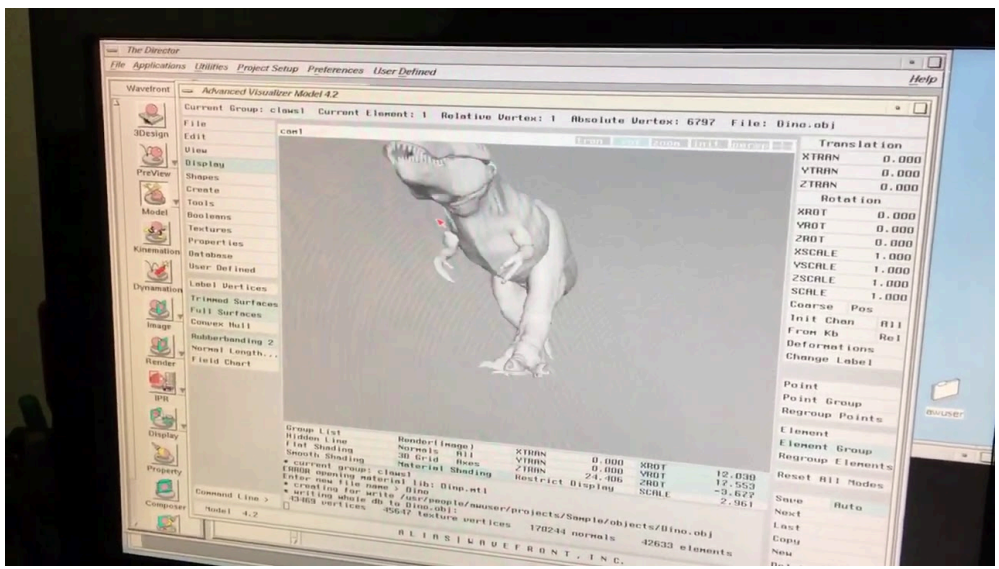
Let's briefly cover some ancient history...

Obj file history

This file format was created by Wavefront Technologies, for their "The Advanced Visualizer" (TAV) 3D graphics production toolkit.

TAV was used in a number of movies, including The Abyss, Terminator 2, and Jurassic Park.

It was software for Silicon Graphics computers, and ran on their version of Unix (Irix).



SGI computers

SGI was Silicon Graphics Interactive. An influential company that made special workstations that could do realtime 3D.

Their workstations ran their own Unix called Irix, and had a graphics library called IrixGL, which became OpenGL, WebGPU's predecessor.



Image credited to Shane Battye from here. Fair use.

OBJ Files

Since TAV was software for a unix system, it's not surprising that it followed the unix philosophy.

That is: it was a suite of small programs that interacted using text files.

One of these text file formats was `.obj`, which was used to describe a 3D mesh. It's simple enough for us to use for class!

It's actually still widely used, due to its high compatibility. You can export OBJ files from tools like Blender.

OBJ files (2)

An obj file is a text file. It consists of a list of lines.

If a line starts with #, it is a comment, so ignore it.

If a line starts with v, it's a vertex. The next 3 wspace-separated numbers are treated as the x, y, and z components of the it. For example:

```
v -0.75 -0.75 0
v 0 0.75 0
v 0.75 -0.75 0
```

Defines the vertices of a triangle

OBJ files (3)

If a line starts with `f`, it is a *face*. Usually that is a triangle, but it doesn't have to be. (All of the .obj files I give you will have only triangle faces)

A face is followed by 3 integers, which are the indices of the vertices that form the face. **Important: they start counting at 1, not 0**

Here is a face that makes a triangle out of the verts in the last slide:

```
f 1 2 3
```

Notice, it's not 0, 1, 2. We have to subtract 1 from these values for them to work with WebGPU correctly! (or add an unused vertex to shift them all up by one).

Loading OBJ files

How do we load them? It's not too bad:

- Create arrays for your vertices and indices
- Read the file into a string
- Split the string into lines
- For each line:
  - If the line starts with `v`, read the next 3 numbers and add the data to the vertex buffer.
  - If the line starts with `f`, read the next 3 numbers and add the data to the index buffer. (I'm ignoring non-triangular faces)
  - If it's a comment or blank line, ignore it.
  - Otherwise, it's another kind of attribute. You can ignore it for now.

Other attributes and faces

OBJ files can have more than vertex positions and faces. They can also have texture coordinates (`vt` lines), normals (`vn` lines), and other things.

Annoyingly, they can also make faces from combinations of vertices. So instead of `f 1 2 3` you might see OBJ files that look like `f 1/7/14 2/5/3 3/9/17`, which means "the first point of the face has position 1, texture coordinate 7, and vertex normal 14". This basically forces you to create a unique vertex with those components, either using dynamic programming or falling back to non-indexed drawing.

I won't give you any OBJ files that do that.

Loading a text file

Loading a text file in browser-land requires the `fetch` API like we used for textures.

We need to `await` it, so we need an `async` function:

```
export async function loadObj(url: URL): Promise<SimpleMesh>{
    const response = await fetch(url);
    const blob = await response.blob();
    const text = await blob.text();
  ...
}
```

After the last await, `text` contains the file as a string.

Preparing the data

The OBJ files we're supporting only have positions and faces. We're randomly generating colors so we can actually see the teapot's triangles.

```
const positions: vec3[] = [];
const colors: vec3[] = [];
const indis: number[] = [];
```

Then we're splitting over lines:

```
for (const line of text.split(/\r?\n/)) {
         const l = line.trim(); // remove extra whitespace before and after
        if (l == '') continue; // ignore empty lines
        if (l.startsWith('#')) continue; // ignore comments
        const parts = l.split(/\s+/)
        if (parts.length == 0) continue;
```


Let's talk about that...

Regexes

Regular expressions (regexes) in Javascript are written between `//`.[1]

If you haven't had automata theory yet, regular expressions are ways of describing patterns in text that can be "matched". Like an email address, or a URL, or a phone number.

In our case, we're matching `/\r?\n/`. `\r` means a carriage return, which is an ascii command to reset the "carriage" (column under cursor) back to the start of the line. `\n` in its original ascii meaning only advanced the line, it kept the cursor over the same column.

[1] this might seem weird, but it was popularized by Perl and seems to be a widely used notation.

Regexes (2)

On Unixy systems (such as Linux, MacOS, and BSD), `\n` is all you need.

But on Windows, newlines in text files are written with a `\r\n`.[1]

OBJ files are supposed to be unixy, but since anyone can open up a text editor and write one out, and since I happen to find myself on Windows right now, it seemed prudent to detect both.

`\r?` means "optional carriage return". The question mark makes the string it's attached to optional.

[1] A popular software engineering streamer, ThePrimagen, refers to these as "registered nurse" line endings, as a mnemonic for the order.

Regexes (3)

We used the `split` method, which breaks a string into an array of strings. The regex is used to detect when the current string is finished.

Afterwards, for each line, we split it again based on spaces.

This makes it easier to read the individual components of the line.

If there aren't the right number of elements in the line, we ignore it.

We use the first element in the array to detect what type of line it is.

Loading OBJs (2)

```
switch (parts[0]) {
  case 'v':
    // vertex position line
    positions.push(vec3.fromValues(
      // '!' means "I promise it's not null or undefined"
        Number.parseFloat(parts[1]!),
        Number.parseFloat(parts[2]!),
        Number.parseFloat(parts[3]!),
    ));
    // generate a random color
    colors.push(vec3.fromValues(
      Math.random(), Math.random(), Math.random()));
    break;
```

Here's how we read vertices...

Loading OBJs (3)

We use `Number.parseFloat` to pull the string components and convert them to floats.

If this fails, it will return `NaN` (not a number), so it would be wise to check for that, but I wanted to keep the code short for the slides.

The `!` casts a `something|undefined` into a `something`. In this case, a string, which is fed to `parseFloat`.

Then we generate a random color. We don't have to do this here, we could actually do it in the vertex shader, but it's a little janky (no built-in random number generator).

Loading OBJs (4)

```
case 'f': // only triangle supported
  indis.push(
    Number.parseInt(parts[1]!) - 1, // obj files count from 1
    Number.parseInt(parts[2]!) - 1, // webgpu counts from 0
    Number.parseInt(parts[3]!) - 1, // so we subtract 1.
  );
  break
case '#':
  break // comment means do nothing
```

Last wrinkles: we use `parseInt` instead of `parseFloat`. We also subtract by 1, because index `1` in an OBJ file will be called `0` in an index buffer.

How was that?

That's about it. Once we have the vertex data, we can pack it into a buffer, which we do in our `LoadedMesh` class.

At that point, we've got everything we need to draw it on screen!

Homework: you're going to need to load OBJ files for your next project, so be sure the sample works for you. Try loading some of the other OBJ files I've included, such as the Cow, the Bunny, and the Armadillo. You might need to change how they are scaled.

So, what do you think? Any questions?

Questions?